

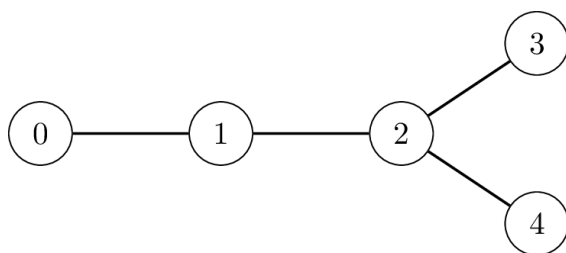
Migrations

Bảo tàng Lịch sử Tự nhiên đang nghiên cứu các mẫu di cư của khủng long ở Bolivia. Các nhà cổ sinh vật học đã phát hiện ra dấu chân khủng long ở N địa điểm khác nhau, được đánh số từ 0 đến $N - 1$ theo thứ tự giảm dần của tuổi: địa điểm 0 chứa các dấu chân lâu đời nhất, trong khi địa điểm $N - 1$ chứa các dấu chân mới nhất.

Khủng long di cư đến từng địa điểm (trừ địa điểm 0) từ một địa điểm cụ thể và lâu đời hơn. Với mỗi địa điểm i sao cho $1 \leq i \leq N - 1$, tồn tại đúng một địa điểm $P[i]$ lâu đời hơn (với $P[i] < i$) sao cho một số loài khủng long di cư trực tiếp từ địa điểm $P[i]$ đến địa điểm i . Một địa điểm lâu đời hơn có thể là nguồn gốc của sự di cư đến nhiều địa điểm mới hơn.

Các nhà cổ sinh vật học mô hình hóa mỗi cuộc di cư như một **kết nối vô hướng** giữa các địa điểm i và $P[i]$. Lưu ý rằng đối với bất kì hai địa điểm khác nhau x và y , khủng long có thể di chuyển từ x đến y bằng cách đi qua một dãy các kết nối. **Khoảng cách** giữa các địa điểm x và y được định nghĩa là số lượng ít nhất các kết nối cần thiết để di chuyển từ x đến y .

Ví dụ, hình ảnh sau đây hiển thị trường hợp có $N = 5$ địa điểm, trong đó $P[1] = 0$, $P[2] = 1$, $P[3] = 2$ và $P[4] = 2$. Khủng long có thể di chuyển từ địa điểm 3 đến địa điểm 4 thông qua 2 kết nối, do đó khoảng cách giữa chúng là 2.



Bảo tàng muốn xác định một cặp địa điểm có khoảng cách xa nhất có thể.

Lưu ý rằng cặp địa điểm cần xác định không nhất thiết phải duy nhất: ví dụ, trong ví dụ trên, cả hai cặp $(0, 3)$ và $(0, 4)$ đều có khoảng cách là 3, là khoảng cách xa nhất. Trong những trường hợp như vậy, bất kì cặp nào đạt khoảng cách xa nhất đều được coi là hợp lệ.

Ban đầu, giá trị của $P[i]$ **không** được biết. Bảo tàng cử một nhóm nghiên cứu lần lượt đến thăm các địa điểm theo thứ tự $1, 2, \dots, N - 1$. Khi đến mỗi địa điểm i ($1 \leq i \leq N - 1$), nhóm thực hiện cả hai hành động sau:

- Xác định giá trị của $P[i]$, nghĩa là nguồn di cư đến địa điểm i .
- Quyết định có gửi **một** tin nhắn tới Bảo tàng hay không gửi bất kì tin nhắn nào từ địa điểm này, dựa trên thông tin đã thu thập trước đó.

Tin nhắn được truyền qua hệ thống liên lạc vệ tinh đắt tiền, vì vậy mỗi tin nhắn phải là một số nguyên từ 1 đến 20 000, bao gồm cả hai giá trị đầu mút. Thêm nữa, nhóm được phép gửi tổng cộng **tối đa 50 tin nhắn**.

Nhiệm vụ của bạn là triển khai một chiến lược, thông qua đó:

- Nhóm nghiên cứu lựa chọn các địa điểm để gửi tin nhắn, cùng với giá trị của mỗi tin nhắn.
- Bảo tàng có thể xác định một cặp địa điểm có khoảng cách xa nhất, chỉ dựa trên tin nhắn nhận được từ mỗi địa điểm, biết được các tin nhắn này được gửi từ địa điểm nào.

Việc gửi các số lớn qua vệ tinh là rất tốn kém. Điểm của bạn sẽ phụ thuộc vào cả số nguyên lớn nhất được gửi và tổng số tin nhắn được truyền đi.

Chi tiết cài đặt

Bạn cần cài đặt hai hàm; một hàm cho nhóm nghiên cứu, và hàm còn lại cho Bảo tàng.

Hàm bạn cần cài đặt cho **nhóm nghiên cứu** là:

```
int send_message(int N, int i, int Pi)
```

- N : số lượng địa điểm với dấu chân khủng long.
- i : chỉ số của địa điểm mà nhóm nghiên cứu đang thăm.
- Pi : giá trị của $P[i]$.
- Hàm này gọi $N - 1$ lần cho mỗi trường hợp thử nghiệm, theo thứ tự $i = 1, 2, \dots, N - 1$.

Hàm này cần trả về $S[i]$ chỉ rõ hành động được nhóm nghiên cứu thực hiện tại địa điểm i :

- $S[i] = 0$: nhóm nghiên cứu quyết định không gửi tin nhắn từ địa điểm i .
- $1 \leq S[i] \leq 20\,000$: nhóm nghiên cứu gửi số nguyên $S[i]$ là tin nhắn từ địa điểm i .

Hàm bạn cần cài đặt cho **Bảo tàng** là:

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- S : mảng kích thước N sao cho:
 - $S[0] = 0$.
 - Với mỗi $1 \leq i \leq N - 1$, $S[i]$ là số nguyên được trả về bởi hàm `send_message(N, i, Pi)`.
- Hàm này được gọi đúng một lần đối với mỗi trường hợp thử nghiệm.

Hàm này cần trả về một cặp địa điểm (U, V) với khoảng cách xa nhất.

Trong quá trình chấm điểm thực tế, một chương trình gọi các hàm trên được chạy **đúng hai lần**.

- Trong lần chạy đầu tiên của chương trình:
 - hàm `send_message` được gọi đúng $N - 1$ lần.
 - **Chương trình của bạn có thể lưu trữ và giữ lại thông tin qua nhiều lần gọi liên tiếp.**
 - Các giá trị trả về (mảng S) được lưu trữ bởi hệ thống chấm điểm.
 - Trong một số trường hợp, cách thức hoạt động của trình chấm điểm là **thích ứng**. Điều này có nghĩa là giá trị của $P[i]$ trong lời gọi `send_message` có thể phụ thuộc vào các hành động mà nhóm nghiên cứu đã thực hiện trong các lời gọi trước đó.
- Trong lần chạy thứ hai của chương trình:
 - hàm `longest_path` được gọi đúng một lần. Thông tin duy nhất có sẵn cho `longest_path` từ hệ thống chấm điểm trong lần chạy đầu tiên là mảng S .

Các ràng buộc

- $N = 10\,000$
- $0 \leq P[i] < i$ với mỗi i sao cho $1 \leq i \leq N - 1$.

Các subtask và Cách chấm điểm

Subtask	Điểm	Các ràng buộc bổ sung
1	30	Địa điểm 0 và địa điểm nào đó có khoảng cách xa nhất trong số tất cả các cặp địa điểm.
2	70	Không có ràng buộc nào thêm.

Gọi Z là số nguyên lớn nhất xuất hiện trong mảng S , và gọi M là số lượng tin nhắn được gửi bởi nhóm nghiên cứu.

Trong bất kỳ trường hợp thử nghiệm nào, nếu ít nhất một trong các điều kiện sau xảy ra thì điểm số cho lời giải của bạn sẽ là 0 (được phản hồi là `Output isn't correct` trong CMS):

- Ít nhất một phần tử trong S là không hợp lệ.
- $Z > 20\,000$ hoặc $M > 50$.
- Giá trị trả về của lời gọi hàm `longest_path` là không chính xác.

Trái lại, điểm của bạn cho subtask 1 được tính như sau:

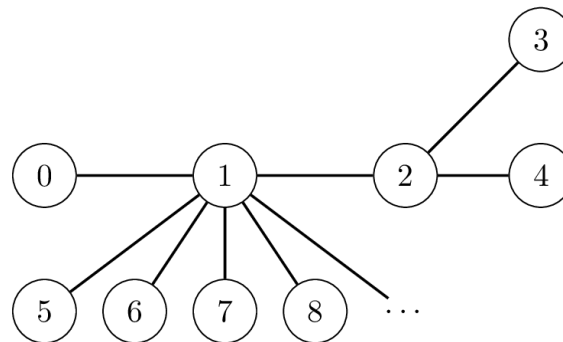
Điều kiện	Điểm
$9\,998 \leq Z \leq 20\,000$	10
$102 \leq Z \leq 9997$	16
$5 \leq Z \leq 101$	23
$Z \leq 4$	30

Điểm cho subtask 2 được tính như sau:

Điều kiện	Điểm
$5 \leq Z \leq 20\,000$ và $M \leq 50$	$35 - 25 \log_{4000} \left(\frac{Z}{5} \right)$
$Z \leq 4$ và $32 \leq M \leq 50$	40
$Z \leq 4$ và $9 \leq M \leq 31$	$70 - 30 \log_4 \left(\frac{M}{8} \right)$
$Z \leq 4$ và $M \leq 8$	70

Ví dụ

Gọi $N = 10\,000$. Xét tình huống mà $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, $P[4] = 2$, và $P[i] = 1$ với $i > 4$.



Giả sử rằng chiến lược của nhóm nghiên cứu là gửi tin nhắn $10 \cdot V + U$ bất cứ khi nào cặp địa điểm (U, V) có khoảng cách xa nhất thay đổi do lời gọi hàm `send_message`.

Ban đầu, cặp có khoảng cách xa nhất là $(U, V) = (0, 0)$. Xét dãy các lời gọi sau đây trong lần chạy đầu tiên của chương trình:

Lời gọi hàm	(U, V)	Giá trị trả về ($S[i]$)
<code>send_message(10000, 1, 0)</code>	$(0, 1)$	10
<code>send_message(10000, 2, 1)</code>	$(0, 2)$	20
<code>send_message(10000, 3, 2)</code>	$(0, 3)$	30
<code>send_message(10000, 4, 2)</code>	$(0, 3)$	0

Lưu ý rằng trong tất cả các lời gọi còn lại, giá trị của $P[i]$ là 1. Điều này có nghĩa là cặp có khoảng cách xa nhất không thay đổi, và nhóm không gửi thêm bất kỳ tin nhắn nào nữa.

Sau đó, trong lần chạy thứ hai của chương trình, lời gọi sau được thực hiện:

```
longest_path([0, 10, 20, 30, 0, ...])
```

Bảo tàng đọc tin nhắn cuối cùng được gửi bởi nhóm nghiên cứu, đó là $S[3] = 30$, và suy ra rằng $(0, 3)$ là cặp điểm có khoảng cách xa nhất. Do đó, lời gọi trả về $(0, 3)$.

Lưu ý rằng cách tiếp cận này không phải lúc nào cũng giúp Bảo tàng xác định chính xác cặp có khoảng cách xa nhất.

Trình chấm mẫu

Trình chấm mẫu gọi cả hàm `send_message` và hàm `longest_path` trong cùng một lần chạy, khác với trình chấm điểm thực tế.

Định dạng dữ liệu vào:

```
N
P[1] P[2] ... P[N-1]
```

Định dạng kết quả ra:

```
S[1] S[2] ... S[N-1]
U V
```

Lưu ý là bạn có thể sử dụng trình chấm mẫu với bất kỳ giá trị nào của N .